Coding Challenge: Finlex Legislation Spider

Objective

Build a Python spider that scrapes Finnish legislation from Finlex for the year **2026**. The spider must discover legislation URLs, extract structured metadata and content from each legislation page, and produce two outputs per document: **cleaned HTML** and **text chunks** (~1000 characters each).

Constraints

- **Python only** (3.10+)
- **HTTP requests only** — use `requests`, `httpx`, or `scrapy`. No browser automation (no Playwright, Selenium, Puppeteer, etc.)
- **No APIs** — scrape the website directly via HTML parsing (e.g., `BeautifulSoup`, `lxml`, `parsel`)
- **Throttle your requests** — add a delay between requests (e.g., 1–2 seconds). Finlex will ban your IP if you send too many requests too fast.
- You may use any open-source libraries available on PyPI
- The spider should be runnable from the command line

Website Overview

Finlex is the official Finnish legislation database. The relevant section is:

- **Landing page**: `https://www.finlex.fi/en/legislation`
- **Year filter**: The landing page has an "Updated statutes by year" sidebar with clickable year buttons (2026, 2025, 2024, ...). Clicking a year filters legislation to that year.
- **Detail pages**: Individual legislation pages (e.g., `https://www.finlex.fi/en/legislation/2017/491`) contain metadata and the full legislative text.

> **Note**: The site is built with Next.js. You'll need to figure out the best strategy to extract content. This is part of the challenge.

> **Important**: Some legislation texts have not been updated yet on the English version. For example, `https://www.finlex.fi/en/legislation/2016/754` shows a notice that the translation is not yet

> available. Your spider must detect this and flag it in the metadata (see `translation_available` field below).

Requirements

1. Discovery — Find All 2026 Legislation

Starting from the legislation landing page, navigate to the 2026 year filter and discover all individual legislation URLs for that year.

- Handle pagination if the listing is paginated
- Collect the full list of legislation URLs before scraping detail pages
- Log how many legislation items were discovered

2. Metadata Extraction

For each legislation page, extract the following metadata into a structured dict/object:

Field	Description	Example
`statute_number`	The statute identifier	`"491/2017"`
`title`	Full title of the legislation	`"Maa- ja metsätalousministeriön asetus..."`
`type_of_statute`	Type classification	`"Decree"`, `"Act"`, etc.
`administrative_sector`	Issuing ministry/body	`"Ministry of Agriculture and Forestry"`
`date_of_issue`	Date the statute was issued	`"25.8.2017"`
`date_of_publication`	Publication date	`"1.9.2017"`
`entry_into_force`	Effective date	`"1.9.2017"`
`language_versions`	Available languages	`["Finnish", "Swedish"]`

Field	Description	Example
`url`	Source URL	`"https://www.finlex.fi/en/legislation/2017/491"`
`status`	Up-to-date / Obsolete	`"Up-to-date"`
`translation_available`	Whether the English text is available	`true` / `false`

If a field is not present on a page, set it to `null`.

3. Content Extraction — Cleaned HTML

Produce a **cleaned HTML** file for each legislation document:

- Keep only the legislative text content (sections, paragraphs, headers)
- Remove navigation, sidebars, footers, metadata tables, and boilerplate
- Preserve the document's semantic structure: headings (`<h1>` – `<h4>`), paragraphs (`<p>`), lists (`<ul>` , `<ol>`), and tables
- The cleaned HTML should be valid, well-formed HTML

4. Chunking

Split each document's text content into **chunks of approximately 1000 characters** (1 token ≈ 1 character):

- Each chunk should be **800–1200 characters** (soft bounds)
- **Never split mid-sentence** — chunk boundaries must fall on sentence boundaries
- **Preserve section context**: each chunk should include a header prefix indicating which section it belongs to (e.g., `"## Section 3 — Definitions\n\n"`)
- If a section is shorter than 800 characters, merge it with the next section
- If a section is longer than 1200 characters, split it at sentence boundaries

Each chunk should be output as a plain text file.

5. Output Format

```
output/
├── metadata.json              # Array of all metadata objects
├── cleaned/
│   ├── 2026_001.html          # Cleaned HTML per statute (use statute number)
│   ├── 2026_002.html
│   └── ...
└── chunks/
```

```
    ├── 2026_001_chunk_001.txt  # Chunks per statute
    ├── 2026_001_chunk_002.txt
    ├── 2026_002_chunk_001.txt
    └── ...
```

Evaluation Criteria

Criterion	Weight	Description
Working spider	30%	Does it successfully discover and scrape 2026 legislation?
Metadata accuracy	20%	Are all metadata fields correctly extracted?
Clean HTML quality	15%	Is boilerplate removed? Is structure preserved?
Chunking logic	20%	Are chunks well-sized, split at sentence boundaries, with section context?
Code quality	15%	Readable, well-structured, error handling, logging

Bonus Points

- **Retry logic** with exponential backoff for failed requests
- **Resumability** — ability to restart without re-scraping already downloaded pages
- **Unit tests** for the metadata extraction and chunking logic

Deliverables

1. Source code (single script or small package)
2. `requirements.txt` or `pyproject.toml` with dependencies
3. Brief `README.md` explaining how to run the spider and any design decisions
4. Sample output from a successful run (at least 5 documents)

Getting Started

```python
# Suggested starting point
pip install requests beautifulsoup4 lxml

# Run
python spider.py --year 2026 --output ./output
```

Good luck!